GradPath AI

University Discovery, Eligibility Rules & AI Research Matching



Submitted By:
Muhammad Talha
2023-CS-12

Submitted To:
Sir Samyan Wahla

Course Name:
Programming for AI

Department of Computer Science
University of Engineering and Technology, Lahore

Contents

1 Executive Summary & Subsystem Boundaries **1**

1.1 Explicit Scope and Boundaries 1

2 Software Architecture & Technology Rationale **2**

2.1 Architectural Style: Layered Modular Monolith 2

3 High-Level Architecture Diagram **3**

4 Implemented Use Cases & Cross-Module Data Contract **4**

4.1 Use Case Implementation Matrix 4

4.2 Downstream Handoff Contract (ApprovedLead v1) 5

5 Software Design Patterns & Mathematical Rationale **6**

5.1 Strategy Pattern (IMatchScoringStrategy → WeightedResearchFitStrategy) 6

5.2 Singleton Pattern (CrawlPolicyRegistry & Serilog Root Pipeline) 7

5.3 Factory & Adapter Patterns (SourceAdapterFactory & ISourceAdapter) . 7

6 UUID Clarification & Structured Serilog Logging **8**

Chapter 1

Executive Summary & Subsystem Boundaries

GradPath AI helps prospective graduate students discover international master's programs and potential research supervisors backed by verifiable institutional sources, evaluate admission eligibility, and transition approved leads into personalized outreach.

This report documents the software architecture, design patterns, mathematical ranking model, and structured logging implementation of the module (University/Professor Discovery, Responsible Data Ingestion, Eligibility Assessment, and AI Research Matching).

1.1 Explicit Scope and Boundaries

To ensure clean modular isolation and support standalone evaluation, this repository enforces strict architectural boundaries:

- **In Scope :** Standalone authentication for isolated demonstration, curated university/professor catalog management (M2-UC02), allowlisted source ingestion adapters (M2-UC01), deterministic academic eligibility assessment with explicit uncertainty states (M2-UC03), explainable weighted research-fit ranking (M2-UC04), scholarship status inspection (M2-UC05), and student lead approval producing the ApprovedLead v1 contract (M2-UC06).
- **Out of Scope (Delegated to Partners):** Full CV document management , automated professor email drafting/sending , and university portal submission checklists .

Chapter 2

Software Architecture & Technology Rationale

2.1 Architectural Style: Layered Modular Monolith

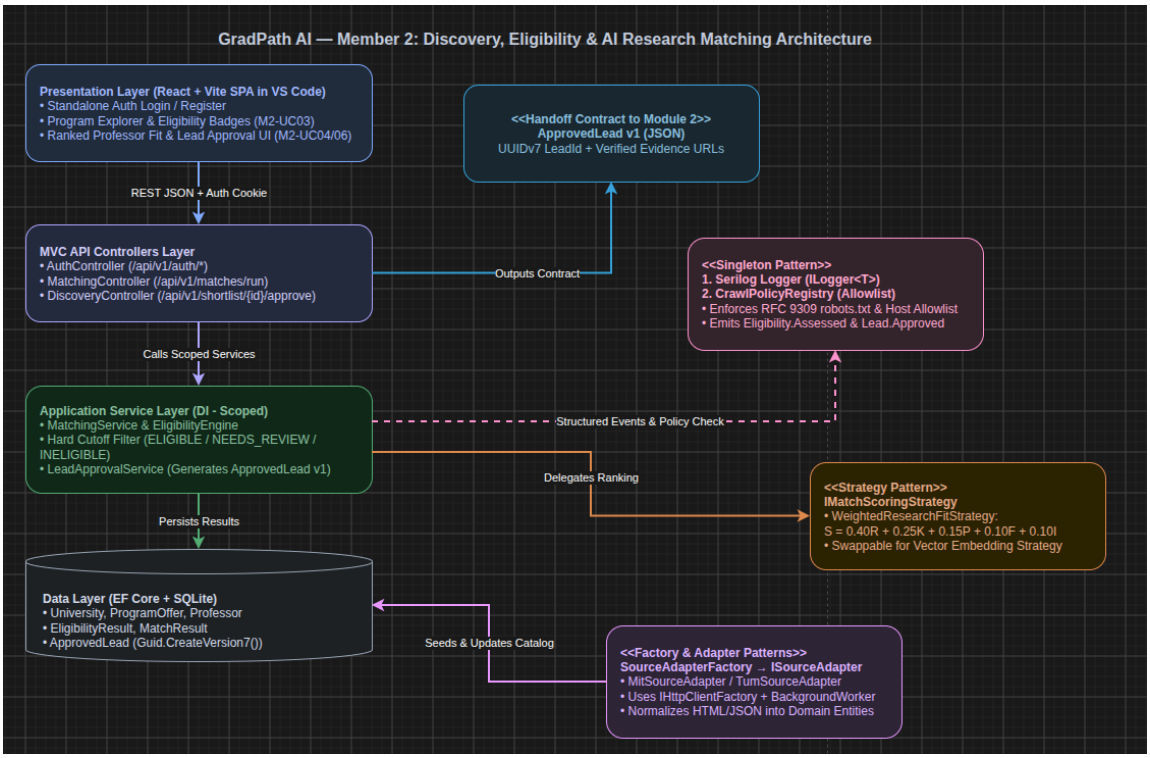
Developed in Visual Studio Code using ASP.NET Core 10 Web API (C#), Entity Framework Core, and a React (Vite) SPA, this subsystem implements the Layered Modular Monolith architecture defined in Chapter 3 of the master specification.

1. **Presentation Layer (React SPA):** Renders the student criteria bar, program/professor search cards, deterministic eligibility badges (ELIGIBLE, NEEDS_REVIEW, INELIGIBLE), transparent score breakdown bars, and the lead approval modal.
2. **API Controller Layer (MVC Pattern):** Exposes RESTful JSON endpoints (MatchingController, DiscoveryController, AuthController) that validate incoming criteria, enforce authentication, and return structured DTOs.
3. **Application Business & Strategy Layer:** Houses EligibilityEngine, MatchingService, and the pluggable IMatchScoringStrategy ranking algorithm, separating hard academic constraints from soft research-fit scoring.
4. **Ingestion Adapters & Persistence Layer:** Contains the EF Core database context, institution-specific HTML/JSON parsers (ISourceAdapter), the SourceAdapterFactory, and the Singleton CrawlPolicyRegistry.

Chapter 3

High-Level Architecture Diagram

Figure 1 presents the high-level architecture of the subsystem, illustrating how the core layered flow interacts with the Strategy, Factory, Adapter, and Singleton patterns.



High-Level Architecture Overview & Design Patterns (Discovery, Eligibility & AI Matching).

Chapter 4

Implemented Use Cases & Cross-Module Data Contract

4.1 Use Case Implementation Matrix

UC Code	Use Case Title	Implementation Scope	Primary API Route
Auth	Standalone User Registration & Login	Fully Working: Cookie session authentication for independent repo grading.	`POST /api/v1/auth/login`
M2-UC02/05	Browse Catalog & Verified Scholarships	Fully Working: Displays attributed programs, professors, and funding status.	`GET /api/v1/opportunities`
M2-UC03	Deterministic Eligibility Assessment	Fully Working: Flags ELIGIBLE, NEEDS_REVIEW, or INELIGIBLE.	`POST /api/v1/matches/run`
M2-UC04	Explainable Professor Research Ranking	Fully Working: Computes weighted score S + human-readable explanation.	`POST /api/v1/matches/run`
M2-UC06	Shortlist & Approve Supervisor Lead	Fully Working: Generates ApprovedLead v1 with UUIDv7 key.	`POST /api/v1/shortlist/{id}/approv`
M2-UC01	Responsible Source Catalog Refresh	Architectural Skeleton: Factory + Adapter + Singleton allowlist check.	`POST /api/v1/catalog/refresh`

Table 4.1: Member 2 individual use case inventory and verification status.

4.2 Downstream Handoff Contract (ApprovedLead v1)

When a student reviews a matched professor's research evidence and clicks Approve Lead, the backend generates and persists an ApprovedLead v1 contract for Module 2 (Member 3's Email Outreach engine). This contract bundles the leadId (UUIDv7), studentId, verified professor metadata, university and program details, evidence URLs with verification timestamps, eligibilityStatus, and fundingStatus. Crucially, the system never fabricates unverified professor email addresses.

Chapter 5

Software Design Patterns & Mathematical Rationale

5.1 Strategy Pattern (IMatchScoringStrategy → WeightedResearchFitStrategy)

- **Where Applied:** Injected into MatchingService via `AddScoped<IMatchScoringStrategy, WeightedResearchFitStrategy>()`.
- **Problem Solved:** AI matching algorithms evolve over the project lifecycle—starting with explainable keyword/lexical overlap in the MVP and expanding to semantic vector embeddings in Phase 2. Hardcoding scoring math inside MatchingService violates the Open/Closed Principle.
- **Mathematical Model (Equation 4.1):** The active strategy computes a normalized suitability score $S \in [0, 1]$ across five evidence-backed dimensions:

$$S = 0.40R + 0.25K + 0.15P + 0.10F + 0.10I \tag{5.1}$$

where R represents research keyword overlap between the student and professor, K measures technical skill alignment, P measures degree program fit, F represents scholarship/funding preference alignment, and I reflects target country/intake match. Hard eligibility cutoffs (such as minimum CGPA) are evaluated separately by EligibilityEngine so that an ineligible program is never promoted as eligible due to high keyword overlap.

5.2 Singleton Pattern (CrawlPolicyRegistry & Serilog Root Pipeline)

- **Where Applied:** Registered via `builder.Services.AddSingleton<ICrawlPolicyRegistry, CrawlPolicyRegistry>()` and `Log.Logger`.
- **Problem Solved:** Responsible web data collection requires a single, centralized, thread-safe coordinator to enforce the allowlist of permitted university hosts (e.g., `mit.edu`, `tum.de`, `ox.ac.uk`), track per-host rate limits, and block private/loopback IP addresses (SSRF defense).
- **Why Singleton is Optimal:** A single shared instance guarantees that concurrent requests and background workers obey the exact same host throttling state and log into a unified Serilog pipeline.

5.3 Factory & Adapter Patterns (SourceAdapterFactory & ISourceAdapter)

- **Where Applied:** SourceAdapterFactory selects institution-specific adapters (MitSourceAdapter, TumSourceAdapter) implementing ISourceAdapter.
- **Problem Solved:** Every university structures its department and faculty web pages differently. A monolithic parser quickly breaks whenever one institution changes its HTML layout.
- **Why Factory + Adapter is Optimal:** Each Adapter encapsulates the parsing logic for a single university and normalizes the extracted data into the internal Professor and ProgramOffer entities, while the Factory dynamically instantiates the correct adapter from the allowlisted institution key.

Chapter 6

UUID Clarification & Structured Serilog Logging

In accordance with Chapter 6.2 and Chapter 9.2 of the specification, 128-bit UUIDs (RFC 9562) are generated via `Guid.CreateVersion7()` for time-ordered primary keys (LeadId, CrawlJobId). A UUID is strictly an internal record identifier and log-correlation key—it is not a proxy, anonymity mechanism, or anti-blocking scraper defense.

All services inject `ILogger<T>` backed by Serilog (writing structured events to the Console and rolling files under `logs/member2-log-.txt`):

```
public sealed class MatchingService : IMatchingService
{
    private readonly IMatchScoringStrategy _scoringStrategy;
    private readonly ILogger<MatchingService> _logger;

    public MatchingService(
        IMatchScoringStrategy scoringStrategy,
        ILogger<MatchingService> logger)
    {
        _scoringStrategy = scoringStrategy;
        _logger = logger;
    }

    public MatchEvaluationDto EvaluateProfessorFit(
        StudentCriteriaDto student,
        Professor prof)
    {
        var eligibility = AssessEligibility(student, prof.Program);
        _logger.LogInformation(
            "Eligibility.Assessed StudentId={StudentId} ProgramId={ProgramId} Status={
    Status}",
            student.StudentId, prof.ProgramId, eligibility);
```

```
        var scoreResult = _scoringStrategy.CalculateScore(student, prof);
        _logger.LogInformation(
            "Match.Scored StudentId={StudentId} ProfessorId={ProfId} Score={Score:F2}
    Strategy={Strategy}",
            student.StudentId, prof.Id, scoreResult.TotalScore,
            "WeightedResearchFitV1");

        return new MatchEvaluationDto(
            prof.Id, eligibility, scoreResult.TotalScore,
            scoreResult.Reasons);
    }
}
```